

INDEX

S.No.	Programs	Date	Signature
1.	FCFS – First Come First Serve		
2.	SJF – Shortest Job First (Non-Preemptive)		
3.	SRTF – Shortest Remaining Time First (Preemptive)		
4.	LJF – Longest Job First (Non-Preemptive)		
5.	LRTF – Longest Remaining Time First (Preemptive)		
6.	Round Robin Scheduling		
7.	Priority Scheduling (Non-Preemptive)		
8.	HRRN – Highest Response Ratio Next		
9.	MLQ – Multi-Level Queue		
10.	FIFO – First In First Out Page Replacement		
11.	LRU – Least Recently Used Page Replacement		
12.	MRU – Most Recently Used Page Replacement		
13.	LFU – Least Frequently Used Page Replacement		
14.	Optimal Page Replacement		
15.	Disk FCFS – First Come First Serve		
16.	SSTF – Shortest Seek Time First		

S.No.	Programs	Date	Signature
17.	SCAN – Elevator Algorithm		
18.	C-SCAN – Circular SCAN		
19.	LOOK Disk Scheduling		
20.	C-LOOK – Circular LOOK		

Question 1

FCFS – First Come First Serve

- Processes run in arrival order.
- Simple but may cause the Convoy Effect.

Source Code (C++):

```
#include<iostream>
#include<string>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas  " << endl;
    cout << " Roll : 24BCS061      " << endl;
    cout << " FCFS CPU Scheduling      " << endl;
    cout << "===== " << endl;

    int n;
    cout << "\nEnter number of processes: ";
    cin >> n;

    string name[n];
    int at[n], bt[n], ct[n], tat[n], wt[n];

    for(int i = 0; i < n; i++){
        name[i] = "P" + to_string(i+1);
        cout << "Enter Arrival Time for P" << (i+1) << ": "; cin >> at[i];
        cout << "Enter Burst Time for P" << (i+1) << ": "; cin >> bt[i];
    }

    for(int i = 0; i < n-1; i++)
        for(int j = 0; j < n-i-1; j++)
            if(at[j] > at[j+1]){
                swap(at[j],at[j+1]);
                swap(bt[j],bt[j+1]);
                swap(name[j],name[j+1]);
            }

    int gantt[200], gs[200], ge[200], gc = 0;
    int cur = 0;
    for(int i = 0; i < n; i++){
        if(cur < at[i]) cur = at[i];
        gs[gc]=cur; gantt[gc]=i; ge[gc]=cur+bt[i]; gc++;
        cur  += bt[i];
        ct[i] = cur;
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }
}
```

```

cout << "\n--- Gantt Chart ---\n|";
for(int i=0;i<gc;i++) cout << " " << name[gantt[i]] << " |";
cout << "\n" << gs[0];
for(int i=0;i<gc;i++) cout << "----" << ge[i];
cout << "\n";

cout << "\nProcess  AT   BT   CT   TAT  WT\n";
cout << "-----\n";
float ttat=0, twt=0;
for(int i=0;i<n;i++){
    cout << name[i] << "          "<<at[i]<<"          "<<bt[i]<<"          "
        <<ct[i]<<"          "<<tat[i]<<"          "<<wt[i]<<"\n";
    ttat+=tat[i]; twt+=wt[i];
}
cout << "-----\n";
cout << "Average TAT : " << ttat/n << "\n";
cout << "Average WT  : " << twt/n  << "\n";
return 0;
}

```

Output:

```

=====
Mohd Anas
Roll : 24BCS061
FCFS CPU Scheduling
=====

Enter number of processes: 4
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 5
Enter Arrival Time for P2: 1
Enter Burst Time for P2: 3
Enter Arrival Time for P3: 2
Enter Burst Time for P3: 8
Enter Arrival Time for P4: 3
Enter Burst Time for P4: 2

--- Gantt Chart ---
| P1 | P2 | P3 | P4 |
0----5----8----16----18

Process  AT   BT   CT   TAT  WT
-----
P1       0    5    5    5    0
P2       1    3    8    7    4
P3       2    8   16   14    6
P4       3    2   18   15   13
-----

Average TAT : 10.25
Average WT  : 5.75

```

Question 2

SJF – Shortest Job First (Non-Preemptive)

- Picks arrived process with smallest burst time.
- Runs it completely before choosing again.

Source Code (C++):

```
#include<iostream>
#include<string>
#include<climits>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas  " << endl;
    cout << " Roll : 24BCS061          " << endl;
    cout << " SJF   (Non-Preemptive)      " << endl;
    cout << "===== " << endl;

    int n;
    cout << "\nEnter number of processes: ";
    cin >> n;

    string name[n];
    int at[n], bt[n], ct[n], tat[n], wt[n];
    bool done[n];

    for(int i=0;i<n;i++){
        done[i]=false;
        name[i]="P"+to_string(i+1);
        cout<<"Enter Arrival Time for P"<<(i+1)<<": "; cin>>at[i];
        cout<<"Enter Burst Time for P"<<(i+1)<<": "; cin>>bt[i];
    }

    int gantt[200], gs[200], ge[200], gc=0;
    int cur=0, completed=0;

    while(completed < n){
        int idx=-1, minB=INT_MAX;
        for(int i=0;i<n;i++)
            if(!done[i] && at[i]<=cur && bt[i]<minB)
                { minB=bt[i]; idx=i; }
        if(idx===-1){ cur++; continue; }
        gs[gc]=cur; gantt[gc]=idx;
        cur+=bt[idx]; ge[gc]=cur; gc++;
        ct[idx]=cur; tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx];
        completed++;
    }
```

```

        done[idx]=true; completed++;
    }

    cout<<"\n--- Gantt Chart ---\n|";
    for(int i=0;i<gc;i++) cout<<" "<<name[gantt[i]]<<" |";
    cout<<"\n"<<gs[0];
    for(int i=0;i<gc;i++) cout<<"----"<<ge[i];
    cout<<"\n";

    cout<<"\nProcess  AT   BT   CT   TAT  WT\n";
    cout<<"-----\n";
    float ttat=0,twt=0;
    for(int i=0;i<n;i++){
        cout<<name[i]<<"          "<<at[i]<<"          "<<bt[i]<<"          "
            <<ct[i]<<"          "<<tat[i]<<"          "<<wt[i]<<"\n";
        ttat+=tat[i]; twt+=wt[i];
    }
    cout<<"-----\n";
    cout<<"Average TAT : "<<ttat/n<<"\n";
    cout<<"Average WT  : "<<twt/n <<"\n";
    return 0;
}

```

Output:

```

=====
Mohd Anas
Roll : 24BCS061
SJF (Non-Preemptive)
=====

Enter number of processes: 4
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 6
Enter Arrival Time for P2: 1
Enter Burst Time for P2: 8
Enter Arrival Time for P3: 2
Enter Burst Time for P3: 7
Enter Arrival Time for P4: 3
Enter Burst Time for P4: 3

--- Gantt Chart ---
| P1 | P4 | P3 | P2 |
0----6----9----16----24

Process  AT   BT   CT   TAT  WT
-----
P1       0    6    6    6    0
P2       1    8   24   23   15
P3       2    7   16   14    7
P4       3    3    9    6    3
-----

Average TAT : 12.25
Average WT  : 6.25

```

Question 3

SRTF – Shortest Remaining Time First (Preemptive)

- At every clock tick the process with least remaining time runs.
- A new arrival can preempt the current process.

Source Code (C++):

```
#include<iostream>
#include<string>
#include<climits>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas  " << endl;
    cout << " Roll : 24BCS061      " << endl;
    cout << " SRTF (Preemptive SJF)      " << endl;
    cout << "===== " << endl;

    int n;
    cout << "\nEnter number of processes: ";
    cin >> n;

    string name[n];
    int at[n], bt[n], rem[n], ct[n], tat[n], wt[n];
    bool done[n];

    for(int i=0;i<n;i++){
        done[i]=false;
        name[i]="P"+to_string(i+1);
        cout<<"Enter Arrival Time for P"<<(i+1)<<": "; cin>>at[i];
        cout<<"Enter Burst Time for P"<<(i+1)<<": "; cin>>bt[i];
        rem[i]=bt[i];
    }

    int gantt[1000], gs[1000], ge[1000], gc=0;
    int cur=0, completed=0, prev=-1;

    while(completed < n){
        int idx=-1, minR=INT_MAX;
        for(int i=0;i<n;i++)
            if(!done[i] && at[i]<=cur && rem[i]<minR)
                { minR=rem[i]; idx=i; }
        if(idx==-1){ cur++; continue; }
        if(idx != prev){
```

```

        if(gc>0) ge[gc-1]=cur;
        gs[gc]=cur; gantt[gc]=idx; gc++;
        prev=idx;
    }
    rem[idx]--; cur++;
    if(rem[idx]==0){
        ge[gc-1]=cur; ct[idx]=cur;
        tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx];
        done[idx]=true; completed++;
    }
}

cout<<"\n--- Gantt Chart ---\n|";
for(int i=0;i<gc;i++) cout<<" "<<name[gantt[i]]<<" |";
cout<<"\n"<<gs[0];
for(int i=0;i<gc;i++) cout<<"--"<<ge[i];
cout<<"\n";

cout<<"\nProcess  AT   BT   CT   TAT  WT\n";
cout<<"-----\n";
float ttat=0,twt=0;
for(int i=0;i<n;i++){
    cout<<name[i]<<"          "<<at[i]<<"          "<<bt[i]<<"          "
        <<ct[i]<<"          "<<tat[i]<<"          "<<wt[i]<<"\n";
    ttat+=tat[i]; twt+=wt[i];
}
cout<<"-----\n";
cout<<"Average TAT : "<<ttat/n<<"\n";
cout<<"Average WT  : "<<twt/n <<"\n";
return 0;
}

```


Output:

```
=====
Mohd Anas
Roll : 24BCS061
SRTF (Preemptive SJF)
=====

Enter number of processes: 4
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 8
Enter Arrival Time for P2: 1
Enter Burst Time for P2: 4
Enter Arrival Time for P3: 2
Enter Burst Time for P3: 9
Enter Arrival Time for P4: 3
Enter Burst Time for P4: 5

--- Gantt Chart ---
| P1 | P2 | P4 | P1 | P3 |
0--1--5--10--17--26

Process  AT   BT   CT   TAT  WT
-----
P1       0    8   17   17   9
P2       1    4    5    4    0
P3       2    9   26   24   15
P4       3    5   10    7    2
-----

Average TAT : 13.00
Average WT  : 6.50
```

Question 4

LJF – Longest Job First (Non-Preemptive)

- Picks arrived process with largest burst time.
- Opposite of SJF; increases average waiting time.

Source Code (C++):

```
#include<iostream>
#include<string>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas  " << endl;
    cout << " Roll : 24BCS061      " << endl;
    cout << " LJF   (Non-Preemptive)      " << endl;
    cout << "===== " << endl;

    int n;
    cout << "\nEnter number of processes: ";
    cin >> n;

    string name[n];
    int at[n], bt[n], ct[n], tat[n], wt[n];
    bool done[n];

    for(int i=0;i<n;i++){
        done[i]=false;
        name[i]="P"+to_string(i+1);
        cout<<"Enter Arrival Time for P"<<(i+1)<<": "; cin>>at[i];
        cout<<"Enter Burst Time for P"<<(i+1)<<": "; cin>>bt[i];
    }

    int gantt[200], gs[200], ge[200], gc=0;
    int cur=0, completed=0;

    while(completed < n){
        int idx=-1, maxB=-1;
        for(int i=0;i<n;i++)
            if(!done[i] && at[i]<=cur && bt[i]>maxB)
                { maxB=bt[i]; idx=i; }
        if(idx==-1){ cur++; continue; }
        gs[gc]=cur; gantt[gc]=idx;
        cur+=bt[idx]; ge[gc]=cur; gc++;
        ct[idx]=cur; tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx];
        done[idx]=true; completed++;
    }
```

```

cout<<"\n--- Gantt Chart ---\n|";
for(int i=0;i<gc;i++) cout<<" "<<name[gantt[i]]<<" |";
cout<<"\n"<<gs[0];
for(int i=0;i<gc;i++) cout<<"----"<<ge[i];
cout<<"\n";

cout<<"\nProcess  AT    BT    CT    TAT  WT\n";
cout<<"-----\n";
float ttat=0,twt=0;
for(int i=0;i<n;i++){
    cout<<name[i]<<"        "<<at[i]<<"        "<<bt[i]<<"        "
        <<ct[i]<<"        "<<tat[i]<<"        "<<wt[i]<<"\n";
    ttat+=tat[i]; twt+=wt[i];
}
cout<<"-----\n";
cout<<"Average TAT : "<<ttat/n<<"\n";
cout<<"Average WT  : "<<twt/n <<"\n";
return 0;
}

```

Output:

```

=====
Mohd Anas
Roll : 24BCS061
LJF (Non-Preemptive)
=====

Enter number of processes: 4
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 4
Enter Arrival Time for P2: 1
Enter Burst Time for P2: 9
Enter Arrival Time for P3: 2
Enter Burst Time for P3: 6
Enter Arrival Time for P4: 3
Enter Burst Time for P4: 2

--- Gantt Chart ---
| P1 | P2 | P3 | P4 |
0----4----13----19----21

Process  AT    BT    CT    TAT  WT
-----
P1       0     4     4     4    0
P2       1     9    13    12    3
P3       2     6    19    17    11
P4       3     2    21    18    16
-----
Average TAT : 12.75
Average WT  : 7.50

```

Question 5

LRTF – Longest Remaining Time First (Preemptive)

- At every clock tick the process with most remaining time runs.
- Preemptive version of LJF.

Source Code (C++):

```
#include<iostream>
#include<string>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas  " << endl;
    cout << " Roll : 24BCS061      " << endl;
    cout << " LRTF (Preemptive LJF)      " << endl;
    cout << "===== " << endl;

    int n;
    cout << "\nEnter number of processes: ";
    cin >> n;

    string name[n];
    int at[n], bt[n], rem[n], ct[n], tat[n], wt[n];
    bool done[n];

    for(int i=0;i<n;i++){
        done[i]=false;
        name[i]="P"+to_string(i+1);
        cout<<"Enter Arrival Time for P"<<(i+1)<<": "; cin>>at[i];
        cout<<"Enter Burst Time for P"<<(i+1)<<": "; cin>>bt[i];
        rem[i]=bt[i];
    }

    int gantt[1000], gs[1000], ge[1000], gc=0;
    int cur=0, completed=0, prev=-1;

    while(completed < n){
        int idx=-1, maxR=-1;
        for(int i=0;i<n;i++)
            if(!done[i] && at[i]<=cur && rem[i]>maxR)
                { maxR=rem[i]; idx=i; }
        if(idx==-1){ cur++; continue; }
        if(idx != prev){
            if(gc>0) ge[gc-1]=cur;
            gs[gc]=cur; gantt[gc]=idx; gc++;
            prev=idx;
        }
        rem[idx]--; cur++;
    }
```

```

        if(rem[idx]==0){
            ge[gc-1]=cur; ct[idx]=cur;
            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];
            done[idx]=true; completed++;
        }
    }

    cout<<"\n--- Gantt Chart ---\n|";
    for(int i=0;i<gc;i++) cout<<" "<<name[gantt[i]]<<" |";
    cout<<"\n"<<gs[0];
    for(int i=0;i<gc;i++) cout<<"--"<<ge[i];
    cout<<"\n";

    cout<<"\nProcess  AT    BT    CT    TAT  WT\n";
    cout<<"-----\n";
    float ttat=0,twt=0;
    for(int i=0;i<n;i++){
        cout<<name[i]<<"        "<<at[i]<<"        "<<bt[i]<<"        "
            <<ct[i]<<"        "<<tat[i]<<"        "<<wt[i]<<"\n";
        ttat+=tat[i]; twt+=wt[i];
    }
    cout<<"-----\n";
    cout<<"Average TAT : "<<ttat/n<<"\n";
    cout<<"Average WT  : "<<twt/n <<"\n";
    return 0;
}

```

Output:

```
=====
Mohd Anas
Roll : 24BCS061
LRTF (Preemptive LJF)
=====

Enter number of processes: 4
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 4
Enter Arrival Time for P2: 1
Enter Burst Time for P2: 9
Enter Arrival Time for P3: 2
Enter Burst Time for P3: 6
Enter Arrival Time for P4: 3
Enter Burst Time for P4: 2

--- Gantt Chart ---
| P1 | P2 | P3 | P2 | P3 | P2 |
0--1--2--3--6--8--12--14--18--21

Process  AT   BT   CT   TAT  WT
-----
P1       0    4    1    1   -3
P2       1    9   21   20   11
P3       2    6   18   16   10
P4       3    2    8    5    3
-----

Average TAT : 10.50
Average WT  : 5.25
```

Question 6

Round Robin Scheduling

- Each process gets a fixed Time Quantum.
- If not done it goes back to the end of the queue.
- Fair and widely used in time-sharing systems.

Source Code (C++):

```
#include<iostream>
#include<string>
#include<queue>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas  " << endl;
    cout << " Roll : 24BCS061      " << endl;
    cout << " Round Robin Scheduling    " << endl;
    cout << "===== " << endl;

    int n, tq;
    cout << "\nEnter number of processes: "; cin >> n;
    cout << "Enter Time Quantum      : "; cin >> tq;

    string name[n];
    int at[n], bt[n], rem[n], ct[n], tat[n], wt[n];

    for(int i=0;i<n;i++){
        name[i]="P"+to_string(i+1);
        cout<<"Enter Arrival Time for P"<<(i+1)<<": "; cin>>at[i];
        cout<<"Enter Burst Time for P"<<(i+1)<<": "; cin>>bt[i];
        rem[i]=bt[i];
    }

    int gantt[1000], gs[1000], ge[1000], gc=0;
    int cur=0, completed=0;
    queue<int> q;
    bool inQueue[n];
    for(int i=0;i<n;i++) inQueue[i]=false;

    for(int i=0;i<n;i++)
        if(at[i]==0){ q.push(i); inQueue[i]=true; }

    while(completed < n){
        if(q.empty()){ cur++; continue; }
        int idx = q.front(); q.pop();
        int execTime = min(tq, rem[idx]);
        gs[gc]=cur; gantt[gc]=idx;
        cur += execTime; rem[idx] -= execTime;
```

```

    ge[gc]=cur; gc++;
    for(int i=0;i<n;i++)
        if(!inQueue[i] && at[i]<=cur && rem[i]>0)
            { q.push(i); inQueue[i]=true; }
    if(rem[idx]==0){
        ct[idx]=cur; tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx]; completed++;
    } else { q.push(idx); }
}

cout<<"\n--- Gantt Chart ---\n|";
for(int i=0;i<gc;i++) cout<<" "<<name[gantt[i]]<<" |";
cout<<"\n"<<gs[0];
for(int i=0;i<gc;i++) cout<<"--"<<ge[i];
cout<<"\n";

cout<<"\nProcess  AT   BT   CT   TAT  WT\n";
cout<<"-----\n";
float ttat=0,twt=0;
for(int i=0;i<n;i++){
    cout<<name[i]<<"      "<<at[i]<<"      "<<bt[i]<<"      "
        <<ct[i]<<"      "<<tat[i]<<"      "<<wt[i]<<"\n";
    ttat+=tat[i]; twt+=wt[i];
}
cout<<"-----\n";
cout<<"Average TAT : "<<ttat/n<<"\n";
cout<<"Average WT  : "<<twt/n <<"\n";
return 0;
}

```


Output:

```
=====
Mohd Anas
Roll : 24BCS061
Round Robin Scheduling
=====

Enter number of processes: 4
Enter Time Quantum      : 2
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 5
Enter Arrival Time for P2: 1
Enter Burst Time for P2: 4
Enter Arrival Time for P3: 2
Enter Burst Time for P3: 2
Enter Arrival Time for P4: 4
Enter Burst Time for P4: 1

--- Gantt Chart ---
| P1 | P2 | P3 | P1 | P4 | P2 | P1 |
0--2--4--6--8--9--11--12

Process  AT   BT   CT   TAT  WT
-----
P1       0    5   12   12   7
P2       1    4   11   10   6
P3       2    2    6    4    2
P4       4    1    9    5    4
-----

Average TAT : 7.75
Average WT  : 4.75
```

Question 7

Priority Scheduling (Non-Preemptive)

- Each process has a priority number (lower = higher priority).
- Highest priority ready process is picked and runs to completion.

Source Code (C++):

```
#include<iostream>
#include<string>
#include<climits>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas  " << endl;
    cout << " Roll : 24BCS061          " << endl;
    cout << " Priority (Non-Preemptive) " << endl;
    cout << "===== " << endl;

    int n;
    cout << "\nEnter number of processes: "; cin >> n;

    string name[n];
    int at[n], bt[n], pr[n], ct[n], tat[n], wt[n];
    bool done[n];

    for(int i=0;i<n;i++){
        done[i]=false;
        name[i]="P"+to_string(i+1);
        cout<<"Enter Arrival Time for P"<<(i+1)<<": "; cin>>at[i];
        cout<<"Enter Burst Time for P"<<(i+1)<<": "; cin>>bt[i];
        cout<<"Enter Priority for P"<<(i+1)
            <<" (lower = higher priority): "; cin>>pr[i];
    }

    int gantt[200], gs[200], ge[200], gc=0;
    int cur=0, completed=0;

    while(completed < n){
        int idx=-1, minP=INT_MAX;
        for(int i=0;i<n;i++){
            if(!done[i] && at[i]<=cur && pr[i]<minP)
                { minP=pr[i]; idx=i; }
        }
        if(idx!=-1){ cur++; continue; }
        gs[gc]=cur; gantt[gc]=idx;
        cur+=bt[idx]; ge[gc]=cur; gc++;
        ct[idx]=cur; tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx];
        done[idx]=true; completed++;
    }
```

```

}

cout<<"\n--- Gantt Chart ---\n|";
for(int i=0;i<gc;i++) cout<<" "<<name[gantt[i]]<<" |";
cout<<"\n"<<gs[0];
for(int i=0;i<gc;i++) cout<<"----"<<ge[i];
cout<<"\n";

cout<<"\nProcess  AT   BT   PR   CT   TAT  WT\n";
cout<<"-----\n";
float ttat=0,twt=0;
for(int i=0;i<n;i++){
    cout<<name[i]<<"      "<<at[i]<<"      "<<bt[i]<<"      "<<pr[i]
        <<"      "<<ct[i]<<"      "<<tat[i]<<"      "<<wt[i]<<"\n";
    ttat+=tat[i]; twt+=wt[i];
}
cout<<"-----\n";
cout<<"Average TAT : "<<ttat/n<<"\n";
cout<<"Average WT  : "<<twt/n <<"\n";
return 0;
}

```

Output:

```
=====
Mohd Anas
Roll : 24BCS061
Priority (Non-Preemptive)
=====

Enter number of processes: 4
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 4
Enter Priority for P1 (lower = higher priority): 2
Enter Arrival Time for P2: 1
Enter Burst Time for P2: 3
Enter Priority for P2 (lower = higher priority): 1
Enter Arrival Time for P3: 2
Enter Burst Time for P3: 5
Enter Priority for P3 (lower = higher priority): 3
Enter Arrival Time for P4: 3
Enter Burst Time for P4: 2
Enter Priority for P4 (lower = higher priority): 1

--- Gantt Chart ---
| P1 | P2 | P4 | P3 |
0----4----7----9----14

Process  AT   BT   PR   CT   TAT  WT
-----
P1       0    4    2    4    4    0
P2       1    3    1    7    6    3
P3       2    5    3   14   12    7
P4       3    2    1    9    6    4
-----

Average TAT : 7.00
Average WT  : 3.50
```

Question 8

HRRN – Highest Response Ratio Next

- Picks process with highest Response Ratio = $(WT + BT) / BT$.
- Balances short jobs and long-waiting jobs. Non-preemptive.

Source Code (C++):

```
#include<iostream>
#include<string>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas " << endl;
    cout << " Roll : 24BCS061 " << endl;
    cout << " HRRN Scheduling " << endl;
    cout << "===== " << endl;

    int n;
    cout << "\nEnter number of processes: "; cin >> n;

    string name[n];
    int at[n], bt[n], ct[n], tat[n], wt[n];
    bool done[n];

    for(int i=0;i<n;i++){
        done[i]=false;
        name[i]="P"+to_string(i+1);
        cout<<"Enter Arrival Time for P"<<(i+1)<<": "; cin>>at[i];
        cout<<"Enter Burst Time for P"<<(i+1)<<": "; cin>>bt[i];
    }

    int gantt[200], gs[200], ge[200], gc=0;
    int cur=0, completed=0;

    while(completed < n){
        int idx=-1;
        float maxRR=-1;
        for(int i=0;i<n;i++){
            if(!done[i] && at[i]<=cur){
                int wt_i = cur - at[i];
                float rr = (float)(wt_i + bt[i]) / bt[i];
                if(rr > maxRR){ maxRR=rr; idx=i; }
            }
        }
        if(idx===-1){ cur++; continue; }
        gs[gc]=cur; gantt[gc]=idx;
        cur+=bt[idx]; ge[gc]=cur; gc++;
        ct[idx]=cur; tat[idx]=ct[idx]-at[idx];
        completed++;
    }
}
```

```

        wt[idx]=tat[idx]-bt[idx];
        done[idx]=true; completed++;
    }

    cout<<"\n--- Gantt Chart ---\n|";
    for(int i=0;i<gc;i++) cout<<" "<<name[gantt[i]]<<" |";
    cout<<"\n"<<gs[0];
    for(int i=0;i<gc;i++) cout<<"----"<<ge[i];
    cout<<"\n";

    cout<<"\nProcess  AT   BT   CT   TAT  WT\n";
    cout<<"-----\n";
    float ttat=0,twt=0;
    for(int i=0;i<n;i++){
        cout<<name[i]<<"      "<<at[i]<<"      "<<bt[i]<<"      "
            <<ct[i]<<"      "<<tat[i]<<"      "<<wt[i]<<"\n";
        ttat+=tat[i]; twt+=wt[i];
    }
    cout<<"-----\n";
    cout<<"Average TAT : "<<ttat/n<<"\n";
    cout<<"Average WT  : "<<twt/n <<"\n";
    return 0;
}

```

Output:

```
=====
Mohd Anas
Roll : 24BCS061
HRRN Scheduling
=====
```

```
Enter number of processes: 4
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 3
Enter Arrival Time for P2: 2
Enter Burst Time for P2: 6
Enter Arrival Time for P3: 4
Enter Burst Time for P3: 4
Enter Arrival Time for P4: 6
Enter Burst Time for P4: 2
```

--- Gantt Chart ---

```
| P1 | P2 | P4 | P3 |
0----3----9----11----15
```

Process	AT	BT	CT	TAT	WT
P1	0	3	3	3	0
P2	2	6	9	7	1
P3	4	4	15	11	7
P4	6	2	11	5	3

Average TAT : 6.50

Average WT : 2.75

Question 9

MLQ – Multi-Level Queue

- Processes assigned to fixed queues by type.
- High priority queue uses Round Robin (tq=2).
- Low priority queue uses FCFS. High queue always served first.

Source Code (C++):

```
#include<iostream>
#include<string>
#include<queue>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas  " << endl;
    cout << " Roll : 24BCS061          " << endl;
    cout << " MLQ Scheduling          " << endl;
    cout << " Q0=High(RR,tq=2)          " << endl;
    cout << " Q1=Low (FCFS)          " << endl;
    cout << "===== " << endl;

    int n;
    cout << "\nEnter number of processes: "; cin >> n;

    string name[n];
    int at[n], bt[n], rem[n], ql[n], ct[n], tat[n], wt[n];
    bool done[n];

    for(int i=0;i<n;i++){
        done[i]=false;
        name[i]="P"+to_string(i+1);
        cout<<"\nEnter Arrival Time for P"<<(i+1)<<": "; cin>>at[i];
        cout<<"Enter Burst Time for P"<<(i+1)<<": "; cin>>bt[i];
        cout<<"Enter Queue Level for P"<<(i+1)
            <<" (0=High / 1=Low): "; cin>>ql[i];
        rem[i]=bt[i];
    }

    int gantt[1000], gs[1000], ge[1000], gc=0;
    int cur=0, completed=0, tq=2;
    bool inQueue[n];
    for(int i=0;i<n;i++) inQueue[i]=false;

    queue<int> q0, q1;
    for(int i=0;i<n;i++){
        if(at[i]==0){
            if(ql[i]==0){ q0.push(i); inQueue[i]=true; }
            else        { q1.push(i); inQueue[i]=true; }
        }
    }
```



```

    }
}

while(completed < n){
    for(int i=0;i<n;i++){
        if(!inQueue[i] && at[i]<=cur){
            if(q1[i]==0){ q0.push(i); inQueue[i]=true; }
            else { q1.push(i); inQueue[i]=true; }
        }
        int idx=-1; bool isQ0=false;
        if(!q0.empty()){ idx=q0.front(); q0.pop(); isQ0=true; }
        else if(!q1.empty()){ idx=q1.front(); q1.pop(); }
        if(idx!=-1){ cur++; continue; }

        int exec = isQ0 ? min(tq,rem[idx]) : rem[idx];
        gs[gc]=cur; gantt[gc]=idx;
        cur+=exec; rem[idx]-=exec;
        ge[gc]=cur; gc++;

        for(int i=0;i<n;i++){
            if(!inQueue[i] && at[i]<=cur){
                if(q1[i]==0){ q0.push(i); inQueue[i]=true; }
                else { q1.push(i); inQueue[i]=true; }
            }
        }
        if(rem[idx]==0){
            ct[idx]=cur; tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];
            done[idx]=true; completed++;
        } else if(isQ0){ q0.push(idx); }
    }

    cout<<"\n--- Gantt Chart ---\n|";
    for(int i=0;i<gc;i++) cout<<" "<<name[gantt[i]]<<" |";
    cout<<"\n"<<gs[0];
    for(int i=0;i<gc;i++) cout<<"--"<<ge[i];
    cout<<"\n";

    cout<<"\nProcess  AT   BT   Queue CT   TAT  WT\n";
    cout<<"-----\n";
    float ttat=0,twt=0;
    for(int i=0;i<n;i++){
        cout<<name[i]<<"          "<<at[i]<<"          "<<bt[i]
            <<"          Q"<<q1[i]<<"          "<<ct[i]<<"          "<<tat[i]<<"          "<<wt[i]<<"\n";
        ttat+=tat[i]; twt+=wt[i];
    }
    cout<<"-----\n";
    cout<<"Average TAT : "<<ttat/n<<"\n";
    cout<<"Average WT : "<<twt/n <<"\n";
    return 0;
}

```

Output:

```
=====
Mohd Anas
Roll : 24BCS061
MLQ Scheduling
Q0=High(RR,tq=2)
Q1=Low (FCFS)
=====

Enter number of processes: 4

Enter Arrival Time for P1: 0
Enter Burst Time for P1: 5
Enter Queue Level for P1 (0=High / 1=Low): 0

Enter Arrival Time for P2: 1
Enter Burst Time for P2: 4
Enter Queue Level for P2 (0=High / 1=Low): 1

Enter Arrival Time for P3: 2
Enter Burst Time for P3: 3
Enter Queue Level for P3 (0=High / 1=Low): 0

Enter Arrival Time for P4: 3
Enter Burst Time for P4: 2
Enter Queue Level for P4 (0=High / 1=Low): 1

--- Gantt Chart ---
| P1 | P3 | P1 | P2 | P4 |
0--2--5--7--9--13--15

Process  AT   BT   Queue CT   TAT  WT
-----
P1       0    5    Q0    7    7    2
P2       1    4    Q1   13   12    8
P3       2    3    Q0    5    3    0
P4       3    2    Q1   15   12   10
-----
Average TAT : 8.50
Average WT  : 5.00
```

Question 10

FIFO – First In First Out Page Replacement

- Page that came into memory first is replaced first.
- Simple queue-based approach. Can suffer from Belady's Anomaly.

Source Code (C++):

```
#include<iostream>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas " << endl;
    cout << " Roll : 24BCS061 " << endl;
    cout << " FIFO Page Replacement " << endl;
    cout << "===== " << endl;

    int frames, n;
    cout << "\nEnter number of frames : "; cin >> frames;
    cout << "Enter number of pages : "; cin >> n;

    int pages[n];
    cout << "Enter page reference string : ";
    for(int i=0;i<n;i++) cin >> pages[i];

    int frame[frames];
    for(int i=0;i<frames;i++) frame[i]=-1;
    int pageFaults=0, pointer=0;

    cout << "\nPage | Frames | Fault?\n";
    cout << "-----+-----+-----\n";
    for(int i=0;i<n;i++){
        int pg=pages[i]; bool found=false;
        for(int j=0;j<frames;j++){
            if(frame[j]==pg){ found=true; break; }
        }
        if(!found){
            frame[pointer]=pg;
            pointer=(pointer+1)%frames;
            pageFaults++;
        }
        cout << " " << pg << " | ";
        for(int j=0;j<frames;j++){
            cout << (frame[j]==-1 ? " - " : " " +to_string(frame[j])+" ");
        }
        cout << " | " << (found?"No":"YES") << "\n";
    }
    cout<<"\nTotal Page Faults : "<<pageFaults<<"\n";
    cout<<"Total Page Hits : "<<(n-pageFaults)<<"\n";
    return 0;
}
```

Output:

```
=====
Mohd Anas
Roll : 24BCS061
FIFO Page Replacement
=====

Enter number of frames : 3
Enter number of pages : 8
Enter page reference string : 1 2 3 4 1 2 5 1

Page | Frames | Fault?
-----+-----+-----
1 | 1 - - | YES
2 | 1 2 - | YES
3 | 1 2 3 | YES
4 | 4 2 3 | YES
1 | 4 1 3 | YES
2 | 4 1 2 | YES
5 | 5 1 2 | YES
1 | 5 1 2 | No

Total Page Faults : 7
Total Page Hits : 1
```

Question 11

LRU – Least Recently Used Page Replacement

- Replaces the page not used for the longest time.
- Tracks last-used time for each frame.

Source Code (C++):

```
#include<iostream>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas " << endl;
    cout << " Roll : 24BCS061 " << endl;
    cout << " LRU Page Replacement " << endl;
    cout << "===== " << endl;

    int frames, n;
    cout << "\nEnter number of frames : "; cin >> frames;
    cout << "Enter number of pages : "; cin >> n;

    int pages[n];
    cout << "Enter page reference string : ";
    for(int i=0;i<n;i++) cin >> pages[i];

    int frame[frames], lastUsed[frames];
    for(int i=0;i<frames;i++){ frame[i]=-1; lastUsed[i]=-1; }
    int pageFaults=0;

    cout << "\nPage | Frames | Fault?\n";
    cout << "-----+-----+-----\n";
    for(int i=0;i<n;i++){
        int pg=pages[i]; bool found=false; int fi=-1;
        for(int j=0;j<frames;j++){
            if(frame[j]==pg){ found=true; fi=j; break; }
        }
        if(found){ lastUsed[fi]=i; }
        else{
            int ri=-1;
            for(int j=0;j<frames;j++){ if(frame[j]==-1){ ri=j; break; } }
            if(ri==-1){
                int mn=lastUsed[0]; ri=0;
                for(int j=1;j<frames;j++){
                    if(lastUsed[j]<mn){ mn=lastUsed[j]; ri=j; }
                }
            }
            frame[ri]=pg; lastUsed[ri]=i; pageFaults++;
        }
        cout<<" "<<pg<<" | ";
        for(int j=0;j<frames;j++){
            cout<<(frame[j]==-1?" - ":" ")+to_string(frame[j])+" ";
        }
        cout<<" | "<<(found?"No":"YES")<<"\n";
    }
}
```

```

    }
    cout<<"\nTotal Page Faults : "<<pageFaults<<"\n";
    cout<<"Total Page Hits    : "<<(n-pageFaults)<<"\n";
    return 0;
}

```

Output:

```

=====
Mohd Anas
Roll : 24BCS061
LRU Page Replacement
=====

Enter number of frames : 3
Enter number of pages : 8
Enter page reference string : 1 2 3 4 1 2 5 1

Page | Frames | Fault?
-----+-----+-----
1 | 1 - - | YES
2 | 1 2 - | YES
3 | 1 2 3 | YES
4 | 4 2 3 | YES
1 | 4 1 3 | YES
2 | 4 1 2 | YES
5 | 5 1 2 | YES
1 | 5 1 2 | No

Total Page Faults : 7
Total Page Hits   : 1

```

Question 12

MRU – Most Recently Used Page Replacement

- Replaces the page that was used most recently.
- Opposite of LRU. Useful for certain cyclic access patterns.

Source Code (C++):

```
#include<iostream>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas " << endl;
    cout << " Roll : 24BCS061 " << endl;
    cout << " MRU Page Replacement " << endl;
    cout << "===== " << endl;

    int frames, n;
    cout << "\nEnter number of frames : "; cin >> frames;
    cout << "Enter number of pages : "; cin >> n;

    int pages[n];
    cout << "Enter page reference string : ";
    for(int i=0;i<n;i++) cin >> pages[i];

    int frame[frames], lastUsed[frames];
    for(int i=0;i<frames;i++){ frame[i]=-1; lastUsed[i]=-1; }
    int pageFaults=0;

    cout << "\nPage | Frames | Fault?\n";
    cout << "-----+-----+-----\n";
    for(int i=0;i<n;i++){
        int pg=pages[i]; bool found=false; int fi=-1;
        for(int j=0;j<frames;j++){
            if(frame[j]==pg){ found=true; fi=j; break; }
        }
        if(found){ lastUsed[fi]=i; }
        else{
            int ri=-1;
            for(int j=0;j<frames;j++){ if(frame[j]==-1){ ri=j; break; } }
            if(ri==-1){
                int mx=lastUsed[0]; ri=0;
                for(int j=1;j<frames;j++){
                    if(lastUsed[j]>mx){ mx=lastUsed[j]; ri=j; }
                }
            }
            frame[ri]=pg; lastUsed[ri]=i; pageFaults++;
        }
        cout<<" "<<pg<<" | ";
        for(int j=0;j<frames;j++){
            cout<<(frame[j]==-1?" - ":" ")+to_string(frame[j])+" ";
        }
        cout<<" | "<<(found?"No":"YES")<<"\n";
    }
}
```

```

    }
    cout<<"\nTotal Page Faults : "<<pageFaults<<"\n";
    cout<<"Total Page Hits    : "<<(n-pageFaults)<<"\n";
    return 0;
}

```

Output:

```

=====
Mohd Anas
Roll : 24BCS061
MRU Page Replacement
=====

Enter number of frames : 3
Enter number of pages : 8
Enter page reference string : 1 2 3 4 1 2 5 1

Page | Frames | Fault?
-----+-----+-----
1 | 1 - - | YES
2 | 1 2 - | YES
3 | 1 2 3 | YES
4 | 1 2 4 | YES
1 | 1 2 4 | No
2 | 1 2 4 | No
5 | 1 5 4 | YES
1 | 1 5 4 | No

Total Page Faults : 5
Total Page Hits   : 3

```


Question 13

LFU – Least Frequently Used Page Replacement

- Replaces the page with the lowest access frequency.
- Ties broken by evicting least recently used among them.

Source Code (C++):

```
#include<iostream>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas " << endl;
    cout << " Roll : 24BCS061 " << endl;
    cout << " LFU Page Replacement " << endl;
    cout << "===== " << endl;

    int frames, n;
    cout << "\nEnter number of frames : "; cin >> frames;
    cout << "Enter number of pages : "; cin >> n;

    int pages[n];
    cout << "Enter page reference string : ";
    for(int i=0;i<n;i++) cin >> pages[i];

    int frame[frames], freq[frames], lastUsed[frames];
    for(int i=0;i<frames;i++){ frame[i]=-1; freq[i]=0; lastUsed[i]=-1; }
    int pageFaults=0;

    cout << "\nPage | Frames | Freq | Fault?\n";
    cout << "-----+-----+-----+-----\n";
    for(int i=0;i<n;i++){
        int pg=pages[i]; bool found=false; int fi=-1;
        for(int j=0;j<frames;j++){
            if(frame[j]==pg){ found=true; fi=j; break; }
        }
        if(found){ freq[fi]++; lastUsed[fi]=i; }
        else{
            int ri=-1;
            for(int j=0;j<frames;j++){ if(frame[j]==-1){ ri=j; break; } }
            if(ri==-1){
                int mf=freq[0]; ri=0;
                for(int j=1;j<frames;j++){
                    if(freq[j]<mf || (freq[j]==mf && lastUsed[j]<lastUsed[ri]))
                        { mf=freq[j]; ri=j; }
                }
                frame[ri]=pg; freq[ri]=1; lastUsed[ri]=i; pageFaults++;
            }
        }
        cout<<" "<<pg<<" | ";
        for(int j=0;j<frames;j++){
            cout<<(frame[j]==-1?" - ":" "+to_string(frame[j]))+" ";
        }
    }
}
```

```

        cout<<" | ";
        for(int j=0;j<frames;j++)
            cout<<(frame[j]==-1?" - ":" "+to_string(freq[j]))+" ";
        cout<<" | "<<(found?"No":"YES")<<"\n";
    }
    cout<<"\nTotal Page Faults : "<<pageFaults<<"\n";
    cout<<"Total Page Hits   : "<<(n-pageFaults)<<"\n";
    return 0;
}

```

Output:

```

=====
Mohd Anas
Roll : 24BCS061
LFU Page Replacement
=====

Enter number of frames : 3
Enter number of pages : 8
Enter page reference string : 1 2 3 4 1 2 5 1

Page | Frames          | Freq          | Fault?
-----+-----+-----+-----
1 | 1 - -          | 1 - -        | YES
2 | 1 2 -          | 1 1 -        | YES
3 | 1 2 3          | 1 1 1        | YES
4 | 4 2 3          | 1 1 1        | YES
1 | 4 2 1          | 1 1 1        | YES
2 | 4 2 1          | 1 2 1        | No
5 | 5 2 1          | 1 2 1        | YES
1 | 5 2 1          | 1 2 2        | No

Total Page Faults : 6
Total Page Hits   : 2

```

Question 14

Optimal Page Replacement

- Replaces page not used for longest time in future.
- Gives minimum page faults but requires future knowledge.

Source Code (C++):

```
#include<iostream>
#include<climits>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas " << endl;
    cout << " Roll : 24BCS061 " << endl;
    cout << " Optimal Page Replacement " << endl;
    cout << "===== " << endl;

    int frames, n;
    cout << "\nEnter number of frames : "; cin >> frames;
    cout << "Enter number of pages : "; cin >> n;

    int pages[n];
    cout << "Enter page reference string : ";
    for(int i=0;i<n;i++) cin >> pages[i];

    int frame[frames];
    for(int i=0;i<frames;i++) frame[i]=-1;
    int pageFaults=0;

    cout << "\nPage | Frames | Fault?\n";
    cout << "-----+-----+-----\n";
    for(int i=0;i<n;i++){
        int pg=pages[i]; bool found=false;
        for(int j=0;j<frames;j++){
            if(frame[j]==pg){ found=true; break; }
        }
        if(!found){
            int ri=-1;
            for(int j=0;j<frames;j++) if(frame[j]==-1){ ri=j; break; }
            if(ri==-1){
                int far=-1; ri=0;
                for(int j=0;j<frames;j++){
                    int nu=INT_MAX;
                    for(int k=i+1;k<n;k++){
                        if(pages[k]==frame[j]){ nu=k; break; }
                        if(nu>far){ far=nu; ri=j; }
                    }
                }
            }
            frame[ri]=pg; pageFaults++;
        }
    }
}
```

```

        cout<<" "<<pg<<" | ";
        for(int j=0;j<frames;j++)
            cout<<(frame[j]==-1?" - ":" "+to_string(frame[j])+" ");
        cout<<" | "<<(found?"No":"YES")<<"\n";
    }
    cout<<"\nTotal Page Faults : "<<pageFaults<<"\n";
    cout<<"Total Page Hits : "<<(n-pageFaults)<<"\n";
    return 0;
}

```

Output:

```

=====
Mohd Anas
Roll : 24BCS061
Optimal Page Replacement
=====

Enter number of frames : 3
Enter number of pages : 8
Enter page reference string : 1 2 3 4 1 2 5 1

Page | Frames | Fault?
-----+-----+-----
1 | 1 - - | YES
2 | 1 2 - | YES
3 | 1 2 3 | YES
4 | 1 2 4 | YES
1 | 1 2 4 | No
2 | 1 2 4 | No
5 | 1 2 5 | YES
1 | 1 2 5 | No

Total Page Faults : 5
Total Page Hits : 3

```

Question 15

Disk FCFS – First Come First Serve

- Serves disk requests in the order they arrive.
- Simple but not efficient for disk head movement.

Source Code (C++):

```
#include<iostream>
#include<cmath>
using namespace std;

int main(){
    cout << "=====" << endl;
    cout << " Mohd Anas  " << endl;
    cout << " Roll : 24BCS061      " << endl;
    cout << " Disk FCFS Scheduling      " << endl;
    cout << "=====" << endl;

    int n, head;
    cout << "\nEnter number of requests: "; cin >> n;
    cout << "Enter initial head position: "; cin >> head;

    int req[n];
    cout << "Enter requests: ";
    for(int i=0;i<n;i++) cin >> req[i];

    int total=0, current=head;
    cout << "\nSeek Sequence:\n" << current;
    for(int i=0;i<n;i++){
        total += abs(req[i]-current);
        current = req[i];
        cout << " -> " << current;
    }
    cout << "\n\nTotal Seek Time: " << total << " cylinders\n";
    return 0;
}
```

Output:

```
=====
Mohd Anas
Roll : 24BCS061
Disk FCFS Scheduling
=====

Enter number of requests: 7
Enter initial head position: 50
Enter requests: 82 170 43 140 24 16 190

Seek Sequence:
50 -> 82 -> 170 -> 43 -> 140 -> 24 -> 16 -> 190

Total Seek Time: 642 cylinders
```

Question 16

SSTF – Shortest Seek Time First

- Serves the closest request first.
- Reduces seek time but can cause starvation.

Source Code (C++):

```
#include<iostream>
#include<cmath>
using namespace std;

int main(){
    cout << "=====" << endl;
    cout << " Mohd Anas  " << endl;
    cout << " Roll : 24BCS061      " << endl;
    cout << " SSTF Disk Scheduling      " << endl;
    cout << "=====" << endl;

    int n, head;
    cout << "\nEnter number of requests: "; cin >> n;
    cout << "Enter initial head position: "; cin >> head;

    int req[n]; bool done[n];
    cout << "Enter requests: ";
    for(int i=0;i<n;i++){ cin >> req[i]; done[i]=false; }

    int total=0, current=head;
    cout << "\nSeek Sequence:\n" << current;
    for(int i=0;i<n;i++){
        int md=9999, pos=-1;
        for(int j=0;j<n;j++){
            if(!done[j] && abs(req[j]-current)<md)
                { md=abs(req[j]-current); pos=j; }
            total+=md; current=req[pos]; done[pos]=true;
            cout << " -> " << current;
        }
        cout << "\n\nTotal Seek Time: " << total << " cylinders\n";
        return 0;
    }
```

Output:

```
=====
Mohd Anas
Roll : 24BCS061
SSTF Disk Scheduling
=====

Enter number of requests: 7
Enter initial head position: 50
Enter requests: 82 170 43 140 24 16 190

Seek Sequence:
50 -> 43 -> 24 -> 16 -> 82 -> 140 -> 170 -> 190

Total Seek Time: 236 cylinders
```


Question 17

SCAN – Elevator Algorithm

- Head moves in one direction serving requests, then reverses.
- Goes to disk end even if no request there.

Source Code (C++):

```
#include<iostream>
#include<algorithm>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas  " << endl;
    cout << " Roll : 24BCS061      " << endl;
    cout << " SCAN Disk Scheduling      " << endl;
    cout << "===== " << endl;

    int n, head, disk_size;
    cout << "\nEnter number of requests: "; cin >> n;
    cout << "Enter initial head position: "; cin >> head;
    cout << "Enter disk size: "; cin >> disk_size;

    int req[n];
    cout << "Enter requests: ";
    for(int i=0;i<n;i++) cin >> req[i];
    char dir;
    cout << "Enter direction (l/r): "; cin >> dir;

    int arr[n+2];
    for(int i=0;i<n;i++) arr[i]=req[i];
    arr[n]=head; arr[n+1]=(dir=='l'?0:disk_size-1);
    sort(arr,arr+n+2);

    int total=0, current=head, pos=0;
    cout << "\nSeek Sequence:\n" << current;
    for(int i=0;i<n+2;i++) if(arr[i]==head){ pos=i; break; }

    if(dir=='r'){
        for(int i=pos+1;i<n+2;i++){
            total+=abs(arr[i]-current); current=arr[i];
            cout<<" -> "<<current;
        }
        for(int i=pos-1;i>=0;i--){
            total+=abs(arr[i]-current); current=arr[i];
            cout<<" -> "<<current;
        }
    } else {
        for(int i=pos-1;i>=0;i--){
```

```

        total+=abs(arr[i]-current); current=arr[i];
        cout<<" -> "<<current;
    }
    for(int i=pos+1;i<n+2;i++){
        total+=abs(arr[i]-current); current=arr[i];
        cout<<" -> "<<current;
    }
}
cout<<"\n\nTotal Seek Time: "<<total<<" cylinders\n";
return 0;
}

```

Output:

```

=====
Mohd Anas
Roll : 24BCS061
SCAN Disk Scheduling
=====

Enter number of requests: 7
Enter initial head position: 50
Enter disk size: 200
Enter requests: 82 170 43 140 24 16 190
Enter direction (l/r): l

Seek Sequence:
50 -> 43 -> 24 -> 16 -> 0 -> 82 -> 140 -> 170 -> 190

Total Seek Time: 230 cylinders

```

Question 18

C-SCAN – Circular SCAN

- Like SCAN but only serves in one direction.
- Jumps back to start after reaching the end.

Source Code (C++):

```
#include<iostream>
#include<algorithm>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas " << endl;
    cout << " Roll : 24BCS061 " << endl;
    cout << " C-SCAN Disk Scheduling " << endl;
    cout << "===== " << endl;

    int n, head, disk_size;
    cout << "\nEnter number of requests: "; cin >> n;
    cout << "Enter initial head position: "; cin >> head;
    cout << "Enter disk size: "; cin >> disk_size;

    int req[n];
    cout << "Enter requests: ";
    for(int i=0;i<n;i++) cin >> req[i];
    char dir;
    cout << "Enter direction (l/r): "; cin >> dir;

    int arr[n+3];
    for(int i=0;i<n;i++) arr[i]=req[i];
    arr[n]=head; arr[n+1]=0; arr[n+2]=disk_size-1;
    sort(arr,arr+n+3);

    int total=0, current=head, pos=0;
    cout<<"\nSeek Sequence:\n"<<current;
    for(int i=0;i<n+3;i++) if(arr[i]==head){ pos=i; break; }

    if(dir=='r'){
        for(int i=pos+1;i<n+3;i++){
            total+=abs(arr[i]-current); current=arr[i];
            cout<<" -> "<<current;
        }
        current=0; cout<<" -> "<<current;
        for(int i=0;i<pos;i++){
            total+=abs(arr[i]-current); current=arr[i];
            cout<<" -> "<<current;
        }
    } else {
```

```

        for(int i=pos-1;i>=0;i--){
            total+=abs(arr[i]-current); current=arr[i];
            cout<<" -> "<<current;
        }
        current=disk_size-1; cout<<" -> "<<current;
        for(int i=n+2;i>pos;i--){
            total+=abs(arr[i]-current); current=arr[i];
            cout<<" -> "<<current;
        }
    }
    cout<<"\n\nTotal Seek Time: "<<total<<" cylinders\n";
    return 0;
}

```

Output:

```

=====
Mohd Anas
Roll : 24BCS061
C-SCAN Disk Scheduling
=====

Enter number of requests: 7
Enter initial head position: 50
Enter disk size: 200
Enter requests: 82 170 43 140 24 16 190
Enter direction (l/r): r

Seek Sequence:
50 -> 82 -> 140 -> 170 -> 190 -> 199 -> 0 -> 16 -> 24 -> 43

Total Seek Time: 382 cylinders

```

Question 19

LOOK Disk Scheduling

- Like SCAN but reverses at last request instead of disk end.
- More efficient than SCAN.

Source Code (C++):

```
#include<iostream>
#include<algorithm>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas  " << endl;
    cout << " Roll : 24BCS061      " << endl;
    cout << " LOOK Disk Scheduling      " << endl;
    cout << "===== " << endl;

    int n, head;
    cout << "\nEnter number of requests: "; cin >> n;
    cout << "Enter initial head position: "; cin >> head;

    int req[n];
    cout << "Enter requests: ";
    for(int i=0;i<n;i++) cin >> req[i];
    char dir;
    cout << "Enter direction (l/r): "; cin >> dir;

    int arr[n+1];
    for(int i=0;i<n;i++) arr[i]=req[i];
    arr[n]=head;
    sort(arr,arr+n+1);

    int total=0, current=head, pos=0;
    cout<<"\nSeek Sequence:\n"<<current;
    for(int i=0;i<n+1;i++) if(arr[i]==head){ pos=i; break; }

    if(dir=='r'){
        for(int i=pos+1;i<n+1;i++){
            total+=abs(arr[i]-current); current=arr[i];
            cout<<" -> "<<current;
        }
        for(int i=pos-1;i>=0;i--){
            total+=abs(arr[i]-current); current=arr[i];
            cout<<" -> "<<current;
        }
    } else {
        for(int i=pos-1;i>=0;i--){
            total+=abs(arr[i]-current); current=arr[i];
```

```

        cout<<" -> "<<current;
    }
    for(int i=pos+1;i<n+1;i++){
        total+=abs(arr[i]-current); current=arr[i];
        cout<<" -> "<<current;
    }
}
cout<<"\n\nTotal Seek Time: "<<total<<" cylinders\n";
return 0;
}

```

Output:

```

=====
Mohd Anas
Roll : 24BCS061
LOOK Disk Scheduling
=====

Enter number of requests: 7
Enter initial head position: 50
Enter requests: 82 170 43 140 24 16 190
Enter direction (l/r): l

Seek Sequence:
50 -> 43 -> 24 -> 16 -> 82 -> 140 -> 170 -> 190

Total Seek Time: 208 cylinders

```

Question 20

C-LOOK – Circular LOOK

- Like C-SCAN but reverses at last request instead of disk end.
- Combines benefits of LOOK and circular movement.

Source Code (C++):

```
#include<iostream>
#include<algorithm>
using namespace std;

int main(){
    cout << "===== " << endl;
    cout << " Mohd Anas  " << endl;
    cout << " Roll : 24BCS061      " << endl;
    cout << " C-LOOK Disk Scheduling    " << endl;
    cout << "===== " << endl;

    int n, head;
    cout << "\nEnter number of requests: "; cin >> n;
    cout << "Enter initial head position: "; cin >> head;

    int req[n];
    cout << "Enter requests: ";
    for(int i=0;i<n;i++) cin >> req[i];
    char dir;
    cout << "Enter direction (l/r): "; cin >> dir;

    int arr[n+1];
    for(int i=0;i<n;i++) arr[i]=req[i];
    arr[n]=head;
    sort(arr,arr+n+1);

    int total=0, current=head, pos=0;
    cout<<"\nSeek Sequence:\n"<<current;
    for(int i=0;i<n+1;i++) if(arr[i]==head){ pos=i; break; }

    if(dir=='r'){
        for(int i=pos+1;i<n+1;i++){
            total+=abs(arr[i]-current); current=arr[i];
            cout<<" -> "<<current;
        }
        for(int i=0;i<pos;i++){
            total+=abs(arr[i]-current); current=arr[i];
            cout<<" -> "<<current;
        }
    } else {
        for(int i=pos-1;i>=0;i--){
            total+=abs(arr[i]-current); current=arr[i];
```

```

        cout<<" -> "<<current;
    }
    for(int i=n;i>pos;i--){
        total+=abs(arr[i]-current); current=arr[i];
        cout<<" -> "<<current;
    }
}
cout<<"\n\nTotal Seek Time: "<<total<<" cylinders\n";
return 0;
}

```

Output:

```

=====
Mohd Anas
Roll : 24BCS061
C-LOOK Disk Scheduling
=====

Enter number of requests: 7
Enter initial head position: 50
Enter requests: 82 170 43 140 24 16 190
Enter direction (l/r): r

Seek Sequence:
50 -> 82 -> 140 -> 170 -> 190 -> 16 -> 24 -> 43

Total Seek Time: 322 cylinders

```